

Registro de Usuarios y Almacenamiento Seguro de Contraseñas

Resumen

La autenticación de usuarios es un componente fundamental en el desarrollo de aplicaciones web modernas. Implementar un sistema de registro seguro no solo protege la información de tus usuarios, sino que también establece la base para funcionalidades más avanzadas como rutas protegidas y niveles de acceso personalizados. En este contenido, exploraremos cómo crear un punto de entrada para registrar usuarios en una base de datos utilizando Node.js, Prisma ORM y técnicas de encriptación.

¿Cómo implementar un sistema de registro de usuarios seguro?

Cuando desarrollamos aplicaciones web, la seguridad de los datos de nuestros usuarios debe ser una prioridad. Implementar un sistema de registro robusto implica no solo almacenar la información del usuario, sino también proteger datos sensibles como las contraseñas mediante técnicas de encriptación.

Para crear nuestro punto de entrada de registro, necesitamos:

- Configurar una ruta POST para recibir los datos del usuario
- Extraer la información del cuerpo de la solicitud
- Encriptar la contraseña antes de almacenarla
- Guardar los datos en la base de datos
- Devolver una respuesta apropiada

Creando el endpoint de registro

Primero, debemos configurar nuestra ruta POST para recibir los datos del nuevo usuario:

```

app.post('/register', async (req, res) => {
  // Extraer información del cuerpo de la solicitud
  const { email, password, name } = req.body;

  // Encriptar la contraseña
  const hashedPassword = await bcrypt.hash(password, 10);

  // Crear el usuario en la base de datos
  const newUser = await prisma.user.create({
    data: {
      email,
      password: hashedPassword,
      name,
      role: 'user'
    }
  });

  // Enviar respuesta
  res.status(201).json({ message: 'User registered successfully' });
});

```

Este código establece un punto de entrada /register que acepta solicitudes POST con los datos del usuario. **La contraseña se encripta utilizando bcrypt con un factor de costo de 10**, lo que proporciona un buen equilibrio entre seguridad y rendimiento.

Protegiendo las contraseñas con bcrypt

La seguridad de las contraseñas es crucial en cualquier sistema de autenticación. **Nunca debemos almacenar contraseñas en texto plano** en nuestra base de datos. En su lugar, utilizamos algoritmos de hash como bcrypt:

```

const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

```

Debemos importar estas librerías al inicio de nuestro archivo para poder utilizarlas. bcrypt nos permite generar un hash seguro de la contraseña, mientras que JWT (JSON Web Token) lo utilizaremos más adelante para la autenticación.

El factor de costo (en este caso 10) determina cuánto trabajo computacional se requiere para generar el hash. **Un valor más alto proporciona mayor seguridad pero consume más recursos.**

Almacenando usuarios con Prisma ORM

Prisma nos facilita la interacción con la base de datos mediante un ORM (Object-Relational Mapping). Para crear un nuevo usuario, utilizamos el método `create` del modelo `User`:

```
const newUser = await prisma.user.create({
  data: {
    email,
    password: hashedPassword,
    name,
    role: 'user'
  }
});
```

Este código crea un nuevo registro en la tabla de usuarios con los datos proporcionados. **El ID se genera automáticamente** gracias a la configuración del modelo en Prisma.

¿Cómo probar nuestro endpoint de registro?

Una vez implementado el código, es importante verificar que funcione correctamente. Podemos utilizar herramientas como Postman para enviar solicitudes a nuestro endpoint y comprobar las respuestas.

Configurando la solicitud en Postman

Para probar nuestro endpoint de registro:

1. Crear una nueva solicitud POST
2. Establecer la URL (por ejemplo, `http://localhost:3000/register`)
3. En la pestaña "Body", seleccionar "raw" y formato JSON
4. Introducir los datos del usuario:

```
{
  "email": "usuario@ejemplo.com",
  "password": "password123",
  "name": "Usuario Ejemplo"
}
```

5. Enviar la solicitud y verificar la respuesta

Si todo funciona correctamente, deberíamos recibir un código de estado 201 (Created) y un mensaje indicando que el usuario se ha registrado con éxito.

Verificando los datos en la base de datos

Para confirmar que los datos se han almacenado correctamente, podemos crear un endpoint temporal que nos permita listar los usuarios:

```
app.get('/db/users', async (req, res) => {
  const users = await prisma.user.findMany();
  res.json(users);
});
```

Este endpoint debe eliminarse en producción ya que expone información sensible, pero es útil durante el desarrollo para verificar que:

- Los IDs se generan correctamente
- Las contraseñas se almacenan como hashes, no en texto plano
- Los roles se asignan adecuadamente

¿Qué consideraciones de seguridad debemos tener en cuenta?

Implementar un sistema de registro seguro implica más que solo encriptar contraseñas. Algunas consideraciones importantes:

- **Proteger las claves secretas:** El secreto utilizado para generar los hashes debe mantenerse seguro.
- **Eliminar endpoints de depuración:** Rutas como /db/users deben eliminarse en producción.
- **Validar los datos de entrada:** Verificar que los datos proporcionados cumplen con los requisitos (formato de email válido, contraseña suficientemente fuerte, etc.).
- **Implementar límites de intentos:** Prevenir ataques de fuerza bruta limitando los intentos de registro desde una misma IP.

La seguridad es un proceso continuo, no una característica que se implementa una vez y se olvida. Debemos estar constantemente actualizando y mejorando nuestras prácticas de seguridad.

La implementación de un sistema de registro seguro es solo el primer paso en la creación de un sistema de autenticación completo. En próximas etapas, desarrollaremos el login de usuarios y la validación de rutas protegidas mediante tokens JWT.

¿Has implementado sistemas de autenticación en tus proyectos? ¿Qué técnicas de seguridad utilizas para proteger los datos de tus usuarios? Comparte tu experiencia en los comentarios.

